Security Capabilities of Open Source Package Registries

Introduction

This document assesses fifteen registries against a common framework: the OpenSSF Securing Software Repositories WG's *Principles for Package Repository Security* (v0.1, February 2024) (OpenSSF Principles), supplemented by the WG's *Trusted Publishers* guidance (OpenSSF Trusted Publishers) and a set of extended capabilities the Principles do not yet cover. The fifteen are npm, PyPI, NuGet, RubyGems, Packagist, Maven Central, crates.io, hex.pm, the Go module ecosystem (proxy.golang.org / sum.golang.org / pkg.go.dev), Debian, Arch Linux, vcpkg, Homebrew, CRAN, and CPAN. They span three models the framework treats differently: self-service registries that accept maintainer-built artifacts, metadata and source registries that point at or mirror source, and curated repositories that build or review what they distribute.

Executive Summary

Across major open-source package registries, security maturity remains uneven and fragmented. Trusted publishing is still limited to a subset of ecosystems, while phishing-resistant or universally enforced MFA remains inconsistent. Registry-level protections also vary significantly: malware detection, vulnerability visibility, security-event transparency, and standardized malicious-package advisories are not consistently available across ecosystems. The largest tooling gap is downstream vulnerability response: most package-manager CLIs still lack built-in SBOM generation and automated remediation, and Go is the only ecosystem in this assessment with native reachability analysis, leaving most consumers unable to distinguish vulnerabilities that are actually reachable from those that are merely present in a dependency. On the other hand, registries have made substantial progress on account and publishing security. Finally, security maturity is closely tied to *sustainable funding*: corporate-backed registries can fund dedicated engineering, while several foundation and volunteer-operated ecosystems continue to depend on grants, sponsorships, or volunteer capacity, making long-term registry sustainability an increasingly important part of software supply-chain security

---

How to read this

The Principles define 32 capabilities across four tracks, Authentication, Authorization, General Capabilities, and CLI Tooling, and assign each to maturity Level 1, 2, or 3. Tables 1 through 4

cover those tracks, one table per track, with each capability as its own column. Table 5 covers thirteen extended capabilities that fall outside the Principles but shape registry security posture, such as version immutability, reproducible builds, cooldown periods, and maintainer-change visibility. Each capability code (A1, Z4, G8, and so on) is defined in full before the table that uses it, so the tables can be read without the Principles document open.

Rating key:

Symbol	Meaning
✅	Shipped and generally available
◐	Partial: limited scope, opt-in, external tooling, or equivalent by other means
❌	Not present
⧗	Announced, in progress, or committed with a public plan
N/A	Out of scope per the Principles' taxonomy (e.g. no user accounts, no hosted artifacts)

A note on the six curated repositories. Debian, Arch, CRAN, CPAN, vcpkg, and Homebrew are scored against the same columns, with N/A where the Principles' taxonomy excludes them. This understates them: the Principles assume a self-service upload model, and these six substitute human review, maintainer keyrings, and centralized build infrastructure for the account-security controls the framework measures. Their strengths show up in Table 5. Arch gets two rows because the official repositories and the AUR have opposite security models, and averaging them would mislead.

Table 1: Authentication

Applies to package repositories with user accounts (OpenSSF Principles).

Capability definitions (paraphrased from the Principles (OpenSSF Principles)):

- **A1. Email verification (L1).** The repository requires users to verify their email address, so a user who loses account access can recover it.
- **A2. Documented account recovery policy (L1).** The repository publishes how account recovery works, making the process predictable and auditable.
- **A3. Strong MFA available (L1).** The repository supports multi-factor authentication via, at minimum, TOTP. SMS, email, or phone-call MFA alone does not qualify.

- **A4. Notifications for critical account changes (L1).** The repository emails maintainers on security-relevant account changes (password change, MFA disabled), so the owner of a compromised account can detect the compromise.
- **A5. Brute-force prevention (L1).** The repository rate-limits or blocks credential-guessing attempts, including guessing of 2FA codes.
- **A6. Abandoned-domain detection (L2).** To prevent account takeover via domain resurrection, the repository detects abandoned email domains (e.g. via WHOIS) and removes the ability to recover an account through an address on an abandoned domain.
- **A7. Phishing-resistant MFA available (L2).** The repository supports MFA via phishing-resistant means such as WebAuthn/FIDO2 security keys or platform authenticators.
- **A8. MFA required for critical packages (L2).** The repository mandates MFA for maintainers of packages deemed critical (e.g. top packages by downloads).
- **A9. Leaked-credential checks (L2).** The repository checks logins against public breach databases (e.g. Have I Been Pwned) and blocks registration with known-leaked passwords.
- **A10. Passwordless authentication (L3).** The repository supports passwordless login, such as passkeys.
- **A11. MFA required for all maintainers (L3).** MFA is mandatory for any account that can publish.
- **A12. Phishing-resistant MFA required for critical packages (L3).** Maintainers of critical packages must use phishing-resistant MFA; the Principles suggest the repository supply security keys to them.

	Level 1					Level 2				Level 3		
Registry	**A1**	**A2**	**A3**	**A4**	**A5**	**A6**	**A7**	**A8**	**A9**	**A10**	**A11**	**A12**
npm	✅	✅	✅	✅	✅	◐	✅	✅	◐	✅	◐	◐
PyPI	✅	✅	✅	✅	✅	✅	✅	✅	✅	◐	✅	❌
NuGet	✅	✅	✅	✅	✅	◐	✅	✅	✅	✅	✅	◐
RubyGems	✅	✅	✅	✅	✅	◐	✅	✅	◐	◐	⧗	❌
Packagist	✅	◐	✅	◐	✅	❌	❌	❌	◐	❌	⧗	⧗
Maven Central	✅	✅	✅	◐	✅	❌	◐	❌	◐	❌	❌	❌
crates.io	✅	◐	✅	◐	✅	◐	✅	◐	✅	✅	❌	❌
hex.pm	✅	◐	✅	◐	✅	❌	◐	N/A	◐	✅	✅	❌

Go modules	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Debian	✅	✅	◐	✅	✅	N/A	◐	N/A	◐	N/A	✅	◐
Arch (official)	✅	◐	◐	◐	✅	❌	◐	N/A	❌	❌	✅	◐
Arch (AUR)	✅	◐	◐	◐	✅	❌	◐	❌	❌	❌	❌	❌
vcpkg	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Homebrew	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
CRAN	◐	N/A	❌	◐	N/A	❌	❌	❌	❌	❌	❌	❌
CPAN (PAUSE)	✅	◐	❌	◐	◐	❌	❌	❌	❌	❌	❌	❌

vcpkg and Homebrew get N/A across this track because neither operates publishing accounts. Contribution happens by pull request against a GitHub repository (Microsoft C++ Team Blog, Trail of Bits), so GitHub owns account security. Go modules get N/A for the same underlying reason the Principles use them as the taxonomy example (OpenSSF Principles): proxy.golang.org has no user accounts to authenticate, since modules come from version-control hosts and identity lives with the VCS host (Go Blog, proxy.golang.org docs). crates.io delegates login to GitHub OAuth (crates.io Docs), so it inherits its A3/A7/A9/A10 ratings rather than implementing them; A11 gets ❌ because crates.io cannot enforce a setting GitHub controls.

Table 2: Authorization

Applies to repositories with user accounts that accept built packages (OpenSSF Principles).

Capability definitions (OpenSSF Principles):

- **Z1. Package-scoped API keys (L1).** Maintainers can provision API keys scoped to specific packages, so automation does not need account-wide credentials.
- **Z2. Token prefixes (L1).** API tokens carry a repository-specific identifier prefix, which makes leaked tokens recognizable to scanners and humans.
- **Z3. Role-based access control (L2).** The repository supports separate roles for managing users versus publishing packages.
- **Z4. Trusted publishing / OIDC (L3).** The repository provisions short-lived API tokens via an OpenID Connect token exchange with a CI identity provider, removing the need for long-lived stored credentials (OpenSSF Trusted Publishers).

- **Z5. Secret-scanning integration with auto-revocation (L3).** Repository tokens are enrolled in third-party secret scanning programs (a known prefix reduces false positives), and the repository operates a reporting endpoint that revokes leaked credentials on report.
- **Z6. Build provenance (L3).** The repository supports verifiable build provenance for packages, attesting to the source and build environment an artifact came from.

	Level 1		L 2	Level 3		
Registry	**Z1**	**Z2**	**Z3**	**Z4**	**Z5**	**Z6**
npm	✅	✅	✅	✅	✅	✅
PyPI	✅	✅	✅	✅	✅	✅
NuGet	✅	✅	◐	✅	✅	◐
RubyGems	✅	✅	✅	✅	✅	✅
Packagist	◐	◐	⧗	❌	◐	N/A
Maven Central	✅	◐	◐	❌	◐	◐
crates.io	✅	✅	◐	✅	✅	❌
hex.pm	✅	◐	✅	❌	◐	⧗
Go modules	N/A	N/A	N/A	N/A	N/A	N/A
Debian	N/A	N/A	✅	N/A	N/A	✅
Arch (official)	N/A	N/A	◐	N/A	N/A	◐
Arch (AUR)	◐	N/A	❌	❌	❌	❌
vcpkg	N/A	N/A	N/A	N/A	N/A	❌
Homebrew	N/A	N/A	N/A	N/A	N/A	✅
CRAN	N/A	N/A	❌	❌	❌	❌
CPAN (PAUSE)	❌	❌	✅	❌	❌	◐

Packagist rates its own Z6 as N/A: it hosts metadata, not artifacts, so the build-provenance requirement does not apply today. The team plans to close that gap by hosting immutable

artifacts (Packagist Blog). Go modules get N/A across the track because there is no publish step and no authenticated API: the mirror fetches source from the VCS host, and the WG's Trusted Publishers document names pkg.go.dev as the case where trusted publishing does not apply (OpenSSF Trusted Publishers). Debian earns its Z6 ✅ through `.buildinfo` files, the buildd network, and reproducible-build verification (Debian Wiki, Reproducible Builds project), which meet the intent by different means. CPAN earns its Z3 ✅ through the `06perms.txt` first-come/maintainer/co-maint model, a real per-module authorization system inside otherwise dated infrastructure (CPAN, Golden).

Table 3: General Capabilities

Applies to all package repositories (OpenSSF Principles).

Capability definitions (OpenSSF Principles):

- **G1. Vulnerability disclosure policy (L1).** The repository publishes a policy letting security researchers report vulnerabilities in the repository itself, with legal safe harbor and a ~90-day remediation target.
- **G2. Typosquat prevention (L1).** The repository takes steps against typosquatting of package names: namespacing, similar-name detection, or comparable measures.
- **G3. Unpublish / name-reuse policy (L2).** Published versions of a package cannot be replaced, and if a package is deleted, others cannot reuse the same name and version.
- **G4. Malicious-package reporting (L2).** Users can report a package as suspicious or malicious via the registry UI, the CLI, and an API.
- **G5. Malware detection (L2).** The repository detects malware, e.g. by scanning uploads against known malicious code snippets and hashes.
- **G6. Vulnerability warnings in the UI (L2).** The registry UI warns of known vulnerabilities in packages and dependencies.
- **G7. Periodic security reviews (L3).** The repository undergoes periodic security reviews of its own infrastructure, and may publish remediations.
- **G8. Event transparency log (L3).** The repository publishes a log of security-relevant events for anomaly detection (the Principles cite `replicate.npmjs.com` as the example).
- **G9. Machine-readable malicious-package advisories (L3).** Advisories for malicious packages use a standardized machine-readable format such as the OSV schema, e.g. via the `ossf/malicious-packages` aggregation (ossf/malicious-packages).

	Level 1		Level 2				Level 3		
Registry	**G1**	**G2**	**G3**	**G4**	**G5**	**G6**	**G7**	**G8**	**G9**

npm	✅	◐	✅	✅	✅	✅	◐	◐	✅
PyPI	✅	◐	✅	✅	✅	◐	✅	◐	✅
NuGet	✅	✅	✅	✅	✅	✅	◐	❌	◐
RubyGems	✅	◐	✅	✅	◐	◐	✅	◐	✅
Packagist	✅	✅	✅	✅	✅	✅	✅	✅	◐
Maven Central	✅	✅	✅	◐	◐	◐	◐	❌	◐
crates.io	✅	◐	✅	◐	◐	✅	✅	◐	◐
hex.pm	✅	◐	✅	◐	❌	◐	✅	❌	❌
Go modules	✅	✅	✅	◐	❌	✅	◐	✅	◐
Debian	✅	✅	✅	✅	◐	✅	◐	✅	❌
Arch (official)	✅	✅	◐	✅	❌	✅	❌	◐	❌
Arch (AUR)	✅	❌	◐	✅	❌	◐	❌	◐	❌
vcpkg	✅	✅	✅	✅	◐	◐	◐	✅	❌
Homebrew	✅	✅	✅	✅	◐	◐	✅	✅	❌
CRAN	◐	◐	✅	◐	◐	❌	❌	◐	❌
CPAN (PAUSE)	✅	◐	✅	◐	❌	✅	❌	◐	❌

One caveat on G8. The Principles cite `replicate.npmjs.com` as the exemple, but GitHub deprecated npm's legacy CouchDB-style replication feeds on 29 May 2025 and replaced them with new replication APIs (GitHub Changelog, GitHub Changelog). An event stream still exists; the fully-replicable public database the Principles pointed to does not, so npm scores ◐. The strongest G8 implementation in the set is Go's: sum.golang.org is a Merkle-tree transparency log of module checksums, backed by Trillian, that the toolchain verifies with inclusion and consistency proofs on every fetch, and it has run in production since 2019 (Go Blog, Go proposal 25530). Packagist runs the only purpose-built, public *security-event* transparency log, covering ownership and reference changes rather than content hashes (Packagist Blog). Debian, vcpkg and Homebrew earn ✅ by a different route: their full change history lives in a public git repository, which serves the same function.

Go's G9 ◐ reflects a split: the Go vulnerability database at vuln.go.dev is OSV-format and CC-BY licensed (golang/vuln), but the Go security team curates it for vulnerabilities, and malicious-module advisories flow mainly through the separate `ossf/malicious-packages` aggregation (ossf/malicious-packages).

Table 4: CLI Tooling

Applies to all ecosystems; scored on the dominant CLI, not the registry (OpenSSF Principles).

Capability definitions (OpenSSF Principles):

- **C1. Pinned installs (L1).** The CLI can install dependencies pinned by hash, version, or lockfile, making resolution deterministic.
- **C2. Vulnerability warnings on install (L2).** The CLI warns of known vulnerabilities in dependencies at install time.
- **C3. SBOM generation (L3).** The CLI can produce a Software Bill of Materials.
- **C4. Automated remediation (L3).** The CLI can, where possible, remediate known vulnerabilities by upgrading dependencies.
- **C5. Reachability analysis (L3).** Static analysis filters vulnerability findings by whether the vulnerable code path is reachable (the Principles cite Go's `govulncheck`).

	L1	L2	Level 3		
Registry / CLI	**C1**	**C2**	**C3**	**C4**	**C5**
npm (`npm`)	✅	✅	✅	✅	❌
PyPI (`pip`)	✅	❌	◐	❌	❌
NuGet (`dotnet`)	✅	✅	◐	◐	❌
RubyGems (`bundler`)	✅	◐	❌	❌	❌
Packagist (`composer`)	✅	✅	◐	❌	❌
Maven Central (`mvn`/`gradle`)	✅	◐	◐	❌	❌
crates.io (`cargo`)	✅	◐	◐	❌	❌
hex.pm (`mix`)	✅	◐	❌	❌	❌

Go (`go`)	✅	◐	◐	❌	✅
Debian (`apt`)	✅	✅	◐	✅	❌
Arch (`pacman`)	◐	◐	❌	❌	❌
vcpkg (`vcpkg`)	✅	❌	✅	❌	❌
Homebrew (`brew`)	◐	❌	❌	◐	❌
CRAN (`install.packages`)	◐	❌	❌	❌	❌
CPAN (`cpan`/`cpanm`)	◐	◐	❌	❌	❌

C5 has one ✅, and it is the framework's own exemplar: `govulncheck` surfaces only vulnerabilities in functions your code calls, backed by the Go security team's curated database (golang/vuln). The other fourteen ecosystems have shipped nothing comparable, and given the volume of AI-assisted vulnerability reports now landing on maintainers, reachability filtering is the highest-leverage unbuilt capability in the framework for all of them.

The C2 ratings separate the default CLI from the ecosystem's audit tooling. `pip` does not warn on install; `pip-audit` does, as a separate install. `cargo` defers to `cargo-audit`, and `go` defers to `govulncheck`. `composer`, `npm` and `dotnet` warn by default (Microsoft .NET Blog, Packagist Blog), and Composer goes further: it blocks malware-flagged versions at install time (Packagist Blog). npm's SBOM support is `npm sbom` (npm Docs); vcpkg emits SPDX SBOMs per installed package (Microsoft C++ Team Blog); Go's C3 ◐ reflects that compiled binaries embed their full module list retrievable via `go version -m`, with SBOM formats left to third-party tools.

Table 5: Extended capabilities (beyond the Principles)

Capability definitions (author's definitions; these do not appear in the Principles):

- **X1. Signed repository metadata / index integrity.** The package *index* is signed or hash-chained so clients detect tampering and can use untrusted mirrors without trusting them. Distinct from signing individual packages.
- **X2. Published-version immutability.** Once a version is published, no one can replace or rewrite its contents; fixes require a new version.

- **X3. Reproducible builds.** Rebuilding a package from the same source yields a bit-identical artifact, so third parties can verify that binaries match source.
- **X4. Centralized attested build infrastructure.** The registry, not the maintainer, builds the distributed artifacts, so a compromised maintainer machine cannot ship a compromised binary on its own.
- **X5. Quarantine / freeze state.** Administrators can place a suspect package into a state that blocks installation without deleting it, preserving evidence and preventing name reuse during incident response.
- **X6. Archival / deprecation state.** Maintainers can mark a package archived or end-of-life so consumers know it will receive no further updates.
- **X7. Namespace or vendor scoping.** Package names are scoped to a verified owner (domain, vendor prefix, reserved prefix), which blocks squatting on an owner's names by construction.
- **X8. Staged publish with human approval.** Publishes land in a staging state and require human approval, ideally MFA-gated, before becoming installable.
- **X9. Install-time code-execution controls.** The registry or client restricts arbitrary code execution during installation (lifecycle scripts, build hooks), or the package format forbids it.
- **X10. Cooldown / minimum release age.** New versions are not served, or not auto-adopted, until a minimum time has elapsed. This exploits the fact that responders usually catch malicious versions within hours of publication.
- **X11. Human review before inclusion.** A human reviews a package or change before it enters the repository.
- **X12. Maintainer-change visibility.** A consumer can see a signal when the set of people who can publish a package changes, so a silent handoff to a new owner is detectable rather than invisible. (Added from the questions Nesbitt's threat-model questionnaire raises; see "How to read this.")
- **X13. Post-removal downstream notification.** A consumer who already installed a version has a channel that flags it once it is pulled, quarantined, or found malicious, rather than learning nothing after the fact.

Registry	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13
npm	◐	✅	❌	❌	◐	✅	✅	✅	✅	◐	❌	❌	◐
PyPI	◐	✅	❌	❌	✅	✅	◐	❌	◐	◐	❌	❌	❌
NuGet	✅	✅	❌	❌	◐	✅	✅	❌	✅	❌	❌	❌	◐
RubyGems	✅	✅	❌	❌	◐	◐	❌	❌	◐	◐	❌	❌	❌
Packagist	◐	✅	❌	❌	⧗	✅	✅	⧗	✅	⧗	❌	✅	◐

Maven Central	◐	✅	◐	❌	◐	◐	✅	✅	✅	❌	❌	❌	❌
crates.io	⧗	✅	◐	❌	◐	◐	❌	❌	◐	❌	❌	❌	❌
hex.pm	✅	✅	❌	❌	◐	✅	◐	❌	◐	◐	❌	❌	❌
Go modules	✅	✅	◐	N/A	◐	✅	✅	N/A	✅	❌	❌	◐	◐
Debian	✅	✅	✅	✅	✅	✅	✅	✅	◐	✅	✅	◐	◐
Arch (official)	✅	◐	◐	✅	◐	✅	✅	◐	❌	◐	✅	◐	◐
Arch (AUR)	❌	◐	❌	❌	◐	✅	❌	❌	❌	❌	❌	◐	❌
vcpkg	✅	✅	◐	◐	◐	◐	N/A	✅	❌	❌	✅	◐	❌
Homebrew	✅	◐	◐	✅	◐	✅	✅	✅	❌	❌	✅	◐	❌
CRAN	◐	✅	❌	✅	◐	✅	❌	◐	❌	◐	✅	❌	❌
CPAN (PAUSE)	◐	✅	❌	❌	❌	◐	✅	❌	❌	❌	❌	◐	❌

The curated repositories earn their keep here. Debian alone holds ✅ across index signing, immutability, reproducibility, centralized builds, cooldown and human review at the same time (Debian Wiki, Reproducible Builds project). On Tables 1–4 Debian looks unremarkable; on Table 5 it has the strongest supply chain in the set. Go's row is the other standout shape: it collects ✅ on index integrity, immutability, namespace scoping and install-time safety through architecture rather than added features, since the checksum database, immutable mirror cache, URL-based module paths, and script-free `go get` are properties of the module system's design (Go Blog, proxy.golang.org docs).

X9 is moving. npm turned lifecycle scripts off by default in v12 (July 2026) (GitHub Changelog), the first major self-service registry to make install-time code execution opt-in. NuGet has had this property since the format's design: packages carry no install scripts (Microsoft Learn). Go has it too: `go get` fetches and verifies source and executes nothing (Go Blog). Arch/AUR, CRAN, CPAN, Homebrew and vcpkg all execute arbitrary build code by construction, which is why review-before-inclusion carries so much weight for them.

The last two columns are the sparsest in the document, which is the point of including them. On X12, only Packagist ships a purpose-built consumer-visible signal for maintainer changes, through its transparency log (Packagist Blog); the ◐ ratings go to the git-native ecosystems (Go, Debian, both Arch rows, vcpkg, Homebrew, CPAN's `06perms.txt`) where an ownership or committer change is visible in a public history but nothing pushes it to you, and the self-service

registries that keep maintainer lists server-side score ❌ because owner-addition emails go to existing owners, not consumers. On X13, no registry earns a ✅: none proactively tells a consumer that something they installed was later pulled. The ◐ ratings all describe pull-based audit tooling the consumer has to run (`npm audit`, `dotnet list package --vulnerable`, `composer audit`, `govulncheck`, apt/pacman security channels), which flags a known-bad installed version only if someone goes looking. Both columns come from the questions Nesbitt's questionnaire asks and no maturity framework yet scores; they are the clearest v0.2 candidates in Table 5.

Per-registry detail

npm

Model: self-service, accepts pre-built artifacts, GitHub-operated.

npm has hardened faster than any other registry in this set, driven by the September 2025 Shai-Hulud worm (The Hacker News). The sequence, per GitHub's changelogs and community discussions: GitHub disabled classic token creation on 5 November 2025 (GitHub Changelog), began revoking classic tokens on 9 December 2025 (GitHub Community Discussion #179562), and expired the rest by 3 February 2026 (GitHub Changelog, GitHub Community Discussion #178140). Granular access tokens replaced them, capped at 90-day lifetimes and 2FA-enforced by default for write scope, and local publishing moved to two-hour session tokens (GitHub Changelog, GitHub Community Discussion #178140). Trusted publishing (GA July 2025 (GitHub Changelog)) supports GitHub Actions, GitLab CI/CD and CircleCI (npm Docs), including a stage-only publisher mode and a per-package "disallow tokens" setting (npm Docs).

The 2026 changes: staged publishing shipped 22 May (GitHub Changelog). Preventive protection for high-impact accounts landed 25 June, placing an account into a 72-hour read-only state on email change or 2FA-recovery-code use (GitHub Changelog). npm v12 (8 July) turned off lifecycle scripts, git dependencies and remote-URL dependencies by default (GitHub Changelog). Publish-time malware scanning and a dual-use content-declaration requirement shipped 28 July (GitHub Changelog).In September, npm added multiple trusted-publishing configurations per package, blocked staged-release approval until malware scanning completes, exposed staged-release history to maintainers, and introduced stage-only granular access tokens that can submit a release without permission to make it installable. GitHub is closing the 2FA-bypass token escape hatch in stages: sensitive account operations were restricted beginning in August 2026 (GitHub Changelog, GitHub Changelog), direct publishing around January 2027 (GitHub Changelog). npm provenance (Sigstore-backed, SLSA-aligned) has been in place since April 2023 (GitHub), and `npm sbom` since 2023 (npm Docs).

Gaps: No reproducible builds. No registry-side cooldown; Dependabot's three-day default cooldown is client-side (BleepingComputer). Typosquat prevention remains reactive: scoped `@org/` packages exist, but the default namespace is flat.

Funding: GitHub built provenance, SBOM, trusted publishing and the 2025–26 hardening in-house. Among the self-service registries that accept artifacts, npm and NuGet are the two whose security roadmap runs entirely on corporate budget rather than grants.

PyPI

Model: self-service, accepts pre-built artifacts, PSF-operated.

PyPI comes closest to implementing the Principles as written, which fits, since trusted publishing originated here (OpenSSF Trusted Publishers, PyPI Blog). Mandatory 2FA for all users (January 2024) (PyPI Blog); trusted publishing (April 2023) (PyPI Blog); GitHub secret scanning integration with automatic token revocation (2023) (PyPI Blog); PEP 740 digital attestations (November 2024) (PyPI Blog); project quarantine (December 2024) (PyPI Blog); project archival (January 2025) (Trail of Bits); ZIP parser-confusion hardening (August 2025) (PyPI Blog); and domain-resurrection prevention (August 2025) (PyPI Blog). That last one makes PyPI the only registry to implement the Principles' A6 abandoned-domain requirement as specified.

Three 2026 additions. After the March 2026 TeamPCP campaign reached LiteLLM and the Telnyx SDK through a compromised Trivy GitHub Action (Akamai, LiteLLM), and PyPI published its own incident report (PyPI Blog), PyPI began rejecting new files uploaded to releases older than 14 days (merged 8 July 2026, announced 22 July) (PyPI Blog). That closes the "poison a long-stable release" path. PyPI also completed its second external security audit (Trail of Bits, conducted 23 February–20 March 2026, announced 16 April) (PyPI Blog, Socket), and began staging a UI overhaul on TestPyPI from 22 July 2026 to surface security signals: attestation display and clearer quarantine/yanked/archived status labeling (PyPI Blog).

Gaps: package index integrity (PEP 458 / TUF) has stayed open for years (PyPI/Warehouse issue #10672), the longest-running unfinished item in this ecosystem. `pip` still does not warn on known vulnerabilities at install. No transparency log; no staged-publish flow.

Funding: Alpha-Omega funds both PSF security staff roles: Seth Larson as Security Developer-in-Residence (since June 2023) and, as of 2026, Mike Fiedler as PyPI Safety & Security Engineer (PSF Blog, PSF Blog); AWS and Georgetown's CSET funded the Fiedler role in its earlier years (PSF, PyPI Blog). Google's Open Source Security Team paid for trusted publishing and attestations (PyPI Blog, PyPI Blog); the Sovereign Tech Fund part-funded the PEP work (PyPI Blog); the Center for Security and Emerging Technology funded the malware reporting API work (PyPI Blog); Alpha-Omega funded archival and the ZIP-confusion fix (Trail of Bits, PyPI Blog); Open Technology Fund paid for the first audit (PyPI Blog); the Sovereign Tech Agency paid for the second audit, with Alpha-Omega funding the remediation capacity (PyPI

Blog); and the OpenSSF funds the security-signal UI overhaul (PyPI Blog, pypi/warehouse issue #19950).

NuGet

Model: self-service, accepts pre-built artifacts, Microsoft-operated.

Repository signing sets NuGet apart: nuget.org signs each uploaded package with its own certificate, on top of optional author signing with pre-registered certificates (Microsoft Learn). Add mandatory 2FA (reported at 100% publisher adoption) (Microsoft .NET Blog), ID prefix reservation, which causes nuget.org to reject submissions matching a reserved prefix from anyone but the owner (Microsoft Learn), and a validation pipeline that runs malware scanning and structural checks before a package becomes resolvable (DevLeader), and NuGet holds a strong L1/L2 position. Vulnerability warnings reached the tooling with NuGet 6.8 (November 2023) (Microsoft .NET Blog). Trusted publishing shipped September 2025 (Microsoft Learn, Microsoft .NET Blog), and the implementation binds trust policies to immutable GitHub owner/repository IDs to defeat resurrection attacks (Microsoft Learn).

One more property gets less attention than it deserves: NuGet packages cannot execute code at install time (Microsoft Learn). That single format decision removes the attack class that dominates npm, PyPI and AUR incidents.

Gaps: SLSA-style build provenance remains a proposal, open since June 2024 (NuGet/Home issue #13581). No transparency log; no staged publishing.

Funding: Microsoft in-house, the second corporate-funded roadmap alongside npm.

RubyGems

Model: self-service, accepts pre-built artifacts, Ruby Central-operated.

RubyGems moved early on several fronts: MFA required for gems above 180 million total downloads since August 2022 (RubyGems Blog), WebAuthn in the browser (2022) and on the CLI (April 2023) (RubyGems Blog), trusted publishing (December 2023) (RubyGems Blog), compact-index integrity work (February 2024) (rubygems.org PR #4167), a user-visible per-account security audit log (February 2024) (RubyGems Blog), and Sigstore attestations (November 2024) (RubyGems Blog). A gemspec can declare `rubygems_mfa_required`, and RubyGems.org will then reject pushes for that gem from any account without MFA. Enforcement moves outward to maintainers (thoughtbot).

Two 2026 events shape the current picture. A July 2026 advisory disclosed that legacy API keys could leak through a Fastly cache misconfiguration on a deprecated endpoint. A Truffle Security researcher reported it on 6 July; the team fixed the root cause on 9 July and revoked the affected keys at disclosure, committing in the same post to expand the 2FA requirement across

the platform in 2026 (RubyGems Blog). In June 2026, attackers used RubyGems as a dead drop rather than a malware vector: fourteen gems uploaded over 6.5 hours carried exfiltrated credential-vault contents (Mend.io). No registry's threat model covers that use today.

Gaps: MFA still is not required across all maintainers; the 180M-download threshold covers about 370 of 190,000+ gems (thoughtbot). Flat namespace, no staged publishing, no publish-time malware scanning.

Funding: Alpha-Omega has funded Ruby Central's security staffing since December 2023: a Security Engineer in Residence, whose work produced the trusted publishing implementation and the audit remediations, joined in 2025 by a full-time Director of Open Source role (Ruby Central). In 2026 Alpha-Omega expanded this to a team of Security Engineers in Residence focused on verifying vulnerabilities in high-dependency gems before reports reach maintainers (Alpha-Omega). Trusted publishing itself came via an AWS Security Residency (RubyGems Blog); Alpha-Omega paid for the December 2024 security audit (RubyGems Blog).

Packagist

Model: metadata registry over git. Composer pulls artifacts from the source host, not from Packagist.

The framework fits this registry awkwardly, and the Packagist team says so themselves: they read Authorization L3 as inapplicable while they host no artifacts (Packagist Blog). The model has a real upside, too: what you install ties back to the source it came from, with no separately-uploaded build in between (Packagist Blog).

Packagist leads the set on transparency. It runs the only purpose-built public security-event transparency log among these registries (Packagist Blog), recording ownership changes, maintainer additions and removals, and version-reference changes. The log recorded the git-retagging behind the April 2026 intercom-php and May 2026 laravel-lang compromises, which let the team reconstruct each incident timeline (Packagist Blog). Mandatory `vendor/package` namespaces make typosquatting harder than on any flat-namespace registry. The team integrated Aikido malware detection in March 2026 (Packagist Blog, Packagist Blog); Composer 2.10 (June 2026) blocks flagged versions at install and unifies malware, advisory and abandoned-package handling under one dependency-policy framework (Packagist Blog). Stable version immutability shipped July 2026, ending silent rewrites via git re-tagging (Packagist Blog). Earlier work: vulnerability notifications in Composer 2.4 (2022) (Packagist Blog), popular-package takeover prevention (2023) (composer/packagist PR #1374), repo-jacking protection (2024) (composer/packagist PR #1411).

Gaps: MFA is the big one. Not mandatory, no FIDO2, no phishing-resistant option (Packagist Blog). Public commitments: mandatory MFA (org-level first), MFA status surfaced in the transparency log and on profiles, FIDO2-backed staged releases required for large packages,

organizational package ownership, a minimum-release-age Composer policy (gated on the immutability work that just shipped), and eventual artifact hosting with SLSA provenance and Sigstore attestations via trusted publishing, targeting Principles Authorization L3 and SLSA Dependency Track L3/L4 by name (Packagist Blog). No trusted publishing today.

Funding: the Sovereign Tech Agency paid for the transparency log and dependency-policy work, via the PHP Foundation (Packagist Blog); Alpha-Omega paid for the 2024 Composer security audit (Packagist Blog), and the PHP Foundation holds a 2026 Alpha-Omega grant for an AI-security-triage residency parallel to the Rust and Ruby programs (Rust Foundation); Aikido provides the malware feed under a CC-BY license (Packagist Blog); Private Packagist revenue funds most of the ongoing work, joined by a sponsorship program launched July 2026 (Packagist Blog).

Maven Central

Model: self-service with verified namespaces, accepts pre-built artifacts, Sonatype-operated.

Maven Central has the strongest namespace verification in the set: a `groupId` must correspond to a domain you can prove you control, or to an `io.github.<username>` namespace bound to your GitHub identity (Sonatype Central). Sonatype has required PGP signing since the early 2010s and removes or rewrites nothing once published (Sonatype Central). Together these give consumers an uncommon ability: verifying artifacts against publisher keys instead of trusting a registry-side claim.

Sigstore signature validation arrived in the Central Publisher Portal in January 2025. Publishers can upload `.sigstore.json` bundles, the Portal validates them, and Central republishes them next to the artifacts for end users (Sonatype Central). Sigstore signing stays optional; invalid signatures will at some point block publishing, missing ones will not, and Sonatype has said it will not replace PGP, though it may end up requiring both (Sonatype Central). Ecosystem tooling caught up with sigstore-java 1.0 (August 2024) (Sigstore Blog). Deployments through the Portal are staged, and the publisher releases them by hand, a human-approval gate by default.

Gaps: no trusted publishing, no mandatory MFA, no transparency log, no leaked-credential checks, no in-registry vulnerability display (it does in sonatype product). The provenance story rests on signatures rather than builds: the signature says who signed an artifact, and nothing attests to how it was built.

crates.io

Model: self-service, accepts pre-built artifacts, Rust Foundation-operated, GitHub OAuth for identity.

crates.io inherits its authentication posture from GitHub, which grants it phishing-resistant MFA, passkeys and leaked-credential checks at no cost, and also means it cannot force maintainers to enable MFA. Trusted publishing shipped July 2025 for GitHub Actions (Rust Blog) per RFC 3691 (RFC 3691), expanded to GitLab CI/CD in January 2026 (Rust Blog), and picked up two hardenings worth naming: a trusted-publishing-only mode that disables token-based publishing for a crate outright, and removal of `pull_request_target` and `workflow_run` as permitted workflow triggers, both repeat offenders in CI compromises (Rust Blog, Socket). Secret scanning integration dates to January 2023 (GitHub Changelog); the team published a threat model in April 2024 (crates.io threat model).

January 2026 also brought a Security tab on crate pages backed by the RustSec advisory database, putting vulnerability data in front of you at dependency-selection time instead of only at audit time (Rust Blog).

Gaps: no build provenance. Artifact signing has sat at the exploration stage since the Rust Foundation's December 2023 announcement (Rust Foundation), and RFC 3724 (TUF for releases and the index) remains open (rust-lang/rfcs PR #3724). The flat namespace is a long-standing structural weakness. No registry-side malware scanning, no cooldown, no staged publishing. These gaps became concrete on 20 August 2026, when compromised maintainer credentials or a compromised device were used to publish malicious versions of arrayref, internment, and append-only-vec together with malicious helper crates. In September, the Rust project also warned maintainers about an ongoing campaign using fake calls and malicious software-install prompts.

Funding: Alpha-Omega has underwritten the Rust Foundation's dedicated security engineering since September 2022, when it and JFrog funded the Foundation's first security team (PR Newswire); the Security Initiative that produced the threat model, trusted publishing, and the Painter and Typomania tooling runs in large part on that support (Rust Foundation). In June 2026 the Foundation added an AI Security Engineer in Residence, Jacob Finkelman, funded through Alpha-Omega's share of the Linux Foundation's $12.5M security pool, to triage AI-generated vulnerability reports before they reach maintainers (Rust Foundation). Alpha-Omega paid for the threat model (crates.io threat model); the OpenSSF funded the Security tab, which Dirkjan Ochtman implemented (Rust Blog).

hex.pm

Model: self-service, accepts pre-built artifacts; serves Erlang, Elixir and Gleam.

Hex jumped forward in 2026. Hex 2.4 (March 2026) replaced password authentication with an OAuth 2.0 device flow, added GitHub sign-in, brought 2FA to the CLI, introduced a sudo mode for sensitive web actions, and made 2FA a prerequisite for OAuth tokens to hold write permissions: without 2FA you get read-only access and cannot publish or manage owners (Hex Blog). In

practice that is mandatory MFA for publishers, reached by a different route than PyPI's. Optional 2FA dates to 2020 (Hex Blog).

Hex has long had something several larger registries lack: a signed registry. Hex.pm signs its registry index, clients verify against published public keys, and the spec states outright that signed registries make untrusted mirrors safe to use (hexpm/specifications, Hex.pm). A clean X1 ✅.

April 2026 brought Hex's first comprehensive third-party security audit, by Paraxial.io and zentrust partners under the Erlang Ecosystem Foundation's Ægis initiative (Hex Blog). The published findings (Paraxial.io) included an OAuth flaw that let a read-only API key escalate to write access given a 2FA code; fixes landed across elixir, rebar3, rebar3_hex, gleam and hex_solver (Paraxial.io). Basic auth and legacy 2FA remain for older clients, slated for phase-out (Hex Blog).

Gaps: no trusted publishing. No attestations or SLSA support; the audit scoped those out as future roadmap (Hex Blog). No malware scanning, no transparency log. `mix hex.audit` reports only retired packages, and advisory scanning falls to the third-party `mix_audit`.

Funding: Alpha-Omega paid for the audit (Hex Blog).

Go modules (proxy.golang.org / sum.golang.org / pkg.go.dev)

Model: source mirror with no accounts and no publish step; Google-operated. Modules live at version-control hosts, the proxy mirrors them, and the checksum database authenticates them.

Go took a different bet than every other registry in this document, and in 2026 the bet looks good. There is nothing to publish and no one to authenticate: a module's name is a URL, releasing means pushing a git tag, and the `go` command fetches through proxy.golang.org, which caches each version immutably, keeping content available even after the origin deletes it (proxy.golang.org docs). The checksum database at sum.golang.org records the hash of each module version in a Merkle-tree transparency log backed by Trillian, and the toolchain checks inclusion and consistency proofs before writing a `go.sum` line (Go Blog, Go proposal 25530). Google, as the operator, cannot falsify or hide entries without detection, which puts Go's index-integrity guarantee ahead of registries that still have TUF on the roadmap. All of this has run in production since Go 1.13 (2019) (Go Blog).

The design closes whole attack classes by construction. There are no publishing credentials to phish, no long-lived tokens to leak, and no install scripts: `go get` executes nothing. Module paths are domain-scoped, so squatting on someone's namespace requires controlling their domain or repository. A maintainer who wants to pull a bad version publishes a `retract` directive rather than deleting anything, and the checksums stay in the log regardless (proxy.golang.org docs). On the vulnerability side, pkg.go.dev displays advisories from the Go

vulnerability database, which the Go security team curates and serves in OSV format at vuln.go.dev under CC-BY (golang/vuln), and `govulncheck` filters findings to functions the code actually calls (golang/vuln), the reachability analysis the Principles cite as their C5 exemplar (OpenSSF Principles) and the only shipped implementation in this set.

Gaps: the model's costs are the mirror image of its strengths. Identity guarantees stop at the VCS host, so a compromised GitHub account compromises the module with no registry-side control in the way; the checksum database proves a version has not changed since first observation, not that the first observation was honest. There is no malware scanning, no formal quarantine state (the proxy can delist, but the mechanism is operational rather than a documented capability), no cooldown, and no reporting API for malicious modules beyond pkg.go.dev's report links. Everything depends on one operator: a single company runs the proxy and the log, and the ecosystem's defense against that concentration is third-party log auditing, which exists but is thin (Valsorda).

Funding: Google corporate budget throughout; the Go security team is Google staff.

Debian

Model: curated distribution. Vetted developers upload signed source; a build network produces binaries; ftpmaster reviews new packages.

Judge Debian on Table 5 rather than Tables 1–4 and it has the strongest supply chain in this document. Developers sign each upload with an OpenPGP key in the Debian keyring, and joining that keyring requires the New Maintainer process. The archive itself is signed (`InRelease`/`Release.gpg`) with keys shipped in `debian-archive-keyring`, and apt verifies by default (Debian Wiki). The buildd network, not developers, produces the binaries. `snapshot.debian.org` preserves the archive's full history. The unstable-to-testing migration delay works as a cooldown, and the NEW queue puts a human in front of anything entering the archive.

The headline 2026 change: as of May 2026, Debian's migration software blocks packages that cannot be rebuilt bit-for-bit, or that regress in reproducibility, from entering testing. Debian 14 "Forky" will be the first major general-purpose distribution to mandate reproducible builds (Reproducible Builds project). Debian sits upstream of 120+ derivatives, so this is the largest reproducibility milestone the ecosystem has had. It also changes post-compromise verification economics, from "rebuild the archive" to "rebuild the packages that fail to match expected hashes."

Gaps: no automated malware scanning; the control is human review, and XZ showed what a patient adversary can do to human review. Maintainer scripts run as root at install time (X9 ◐). Advisories go out as DSAs and CVEs rather than OSV malicious-package records, leaving Debian mostly invisible to OSV-based malicious-package tooling (G9 ❌).

Arch Linux

Model: two models in one ecosystem, scored apart.

The official repositories are curated: Trusted Users and Developers build and sign packages with keys in the `archlinux-keyring`, repository databases are signed, and `pacman` verifies package signatures by default. A solid, smaller-scale version of the Debian model.

The AUR, by declared design philosophy, gets no review: users submit PKGBUILDs, arbitrary build scripts, that other users build on their own machines, with no signing, no review, no malware scanning, no namespace protection. 2026 showed what that costs at scale. In June, the "Atomic Arch" campaign (Sonatype's name for it) backdoored 400+ AUR packages with a Rust infostealer and an eBPF rootkit, spoofing git committer metadata so the changes appeared to come from a long-standing maintainer, and pulling in a malicious npm package (`atomic-lockfile`) whose install hook ran at build time (TheNextWeb, BrainDetox). Then on 30 July 2026, four days before this assessment, the Arch DevOps team disabled the package adoption feature outright, after attackers made a pattern of adopting orphaned packages and injecting malicious follow-up commits. One researcher tracking the campaign counted 200+ affected packages; about 13,000 orphaned packages remain in the AUR (BleepingComputer, TheNextWeb).

The structural problem: the AUR's abandonment surface is its attack surface, and adoption was the mechanism meant to fix abandonment. Turning adoption off trades one failure mode for another. For anyone working on delegated or autonomous maintenance of abandoned packages, this is the live case study: open, unreviewed adoption of trusted names now works as an attack primitive at multi-hundred-package scale.

vcpkg

Model: curated git-based registry of build recipes. Microsoft maintainers review each PR; users build from source.

vcpkg has no accounts, no artifact hosting, and no publishing API, so most of the Principles do not apply. What it does have is strong for a curated registry: SHA512 pins on each upstream download, git-tree object hashes in the versions database (Microsoft Learn), a public git history as audit trail, and CI that validates each port change by rebuilding all dependent and depended-upon ports across the main triplets before merge (Microsoft C++ Team Blog). About 2,849 ports as of June 2026 (Microsoft C++ Team Blog). It generates SPDX SBOMs per installed package, ahead of most self-service registries on C3. In early 2026 the team revised the maintainer guide with written policies on port maturity, project activity, naming, and PR review expectations, codifying the human-review gate that does the real security work (Microsoft C++ Team Blog).

Gaps: portfiles are CMake scripts running arbitrary build logic, so a malicious port is a code-execution primitive, and PR review is the whole defense. No provenance for user-built binaries, no in-CLI vulnerability warnings, no cooldown.

Homebrew

Model: curated formulae, centralized build infrastructure. Homebrew builds all bottles itself.

The Principles use Homebrew as their example of a repository that builds on behalf of users (OpenSSF Principles), and the model does real work. Homebrew's CI, not contributors, builds the bottles, and since 2024 each bottle build emits a build-provenance attestation via GitHub Artifact Attestations, backed by Sigstore's transparency log (Homebrew Docs, Trail of Bits). Set `HOMEBREW_VERIFY_ATTESTATIONS` and `brew` verifies provenance before installation; a "backfill" attestation covers pre-existing bottles, so an attacker cannot downgrade verification by omitting an attestation (Homebrew Docs, Trail of Bits, Homebrew/brew issue #17019). Verification is not yet the default; the blocker is a dependency on the `gh` CLI, pending a pure-Ruby implementation (Homebrew Docs). The design went through the OpenSSF WG (OpenSSF WG proposal).

Building from source has already paid off once: Homebrew's own security documentation notes that in the 2026 Trivy compromise, the homebrew/core formula was unaffected because it built from source rather than from the retagged release binaries (Homebrew Docs).

Gaps: weak pinning; `brew` moves formulae forward and pinning fights the tool (C1 ◐). Formulae are Ruby evaluated at runtime (X9 ❌). Attestation verification off by default. Third-party taps sit outside all of these guarantees, and Homebrew warns loudly when you use them (Homebrew Docs).

Funding: build provenance came out of an Alpha-Omega and Trail of Bits collaboration (Trail of Bits, project announcement); Open Technology Fund paid for the 2024 security audit (Homebrew).

CRAN

Model: curated, email-submission based, with centralized binary builds for Windows and macOS.

CRAN's security model is the submission gate, and little else. Each package must pass `R CMD check`: installation alongside the rest of CRAN, unit tests, documentation and URL checks, plus human review by the CRAN team. The repository policy prohibits circumventing security provisions such as TLS verification (CRAN). Packages failing checks against new R releases get archived (CRAN). The archive keeps all historical versions, so immutability and archival are

strong. Posit's assessment of the model is candid: the checks "do not specifically target malicious code" but "provide a significant hurdle" (Posit Support).

There is no account system in the modern sense. Identity is the maintainer's email address, confirmed per submission. No MFA, no API tokens, no package signing, no malware scanning, no vulnerability display; vulnerability tooling (`oysteR`) lives outside CRAN.

Assessment: CRAN scores worst of the fifteen against the Principles as written. Part of that is a framework artifact, since the human-review gate has real value and the framework has no column for it (hence X11). Part of it is real: email-based identity with no second factor is the weakest publisher-authentication story in this document.

CPAN / PAUSE

Model: self-service upload to PAUSE, mirrored to CPAN, with a per-module permissions system.

CPAN's good idea is `06perms.txt`: per-module permissions with first-come, maintainer and co-maintainer roles, where an author must hold permissions on each module name they want indexed (CPAN). Closing the distribution-name loophole in this system fixed a real hijacking vector, RT-queue takeover through distribution-name squatting (Golden). BackPAN keeps all uploads forever, so immutability holds by construction.

The infrastructure around it has aged. PAUSE has no 2FA. Signing exists through optional `SIGNATURE` files and PAUSE's PGP-signed `CHECKSUMS`, but the CVE cluster disclosed in 2021 (CVE-2020-16154/16155/16156) showed that `cpan` and `cpanminus` could be tricked past signature verification and that `CPAN::ChecKsums` failed to identify packages within signed data; the standing guidance remains to rely on TLS-protected mirrors rather than signatures (Hackeriet).

The bright spot is governance. The CPAN Security Group became the CVE Numbering Authority for CPAN and Perl in February 2025, and MetaCPAN now displays security advisories on distribution pages (CPAN Security Group). A real G1/G6 improvement, run by volunteers.

Key Gaps Summarized

Control	Assessment
Build provenance	Available in selected ecosystems and typically opt-in; not a universal registry guarantee.

Reproducible builds	Debian is the leading large-scale distribution example, but no ecosystem has complete, universal artifact reproducibility verification.
Publish review / staging	npm has a distinct MFA-gated staged-release workflow; other registries generally rely on direct authorized publication, external CI controls, or repository workflow conventions.
Release cooldown	Rare as a registry default; increasingly available as a consumer-side dependency policy.
MFA enforcement	Strongest at PyPI; NuGet has linked-account enforcement; RubyGems has high-impact-package enforcement plus opt-in mechanisms; policies differ significantly elsewhere.
Ownership-change observability	Current owner/maintainer metadata is commonly available; standardized history, notifications, and consumer-facing risk signals are not.
Reachability-aware vulnerability analysis	`govulncheck` is the benchmark for ecosystem-native implementation; multi-ecosystem tools are emerging, but reachability is not yet a broadly default capability.

Cross-cutting observations

Trusted publishing has become the dividing line. Five of the fifteen support it: npm (GitHub Changelog), NuGet (Microsoft Learn), PyPI (PyPI Blog), RubyGems (RubyGems Blog), and crates.io (Rust Blog); outside this set, pub.dev does too, matching the OpenSSF WG's own support list (OpenSSF Trusted Publishers). Packagist, Maven Central and hex.pm do not, and for Packagist the blocker is architectural (Packagist Blog). Go sits outside the question by design: with no publish step, the WG's own guidance marks trusted publishing inapplicable to it (OpenSSF Trusted Publishers). The Principles place trusted publishing at Authorization L3; in practice it has become the entry ticket for credible CI publishing, and the registries that have it are moving past it toward exclusivity: crates.io's trusted-publishing-only mode (Rust Blog), npm's disallow-tokens setting and stage-only publisher configuration (npm Docs).

The 2025–26 attack wave pushed the roadmaps in one direction. Shai-Hulud (September 2025) (The Hacker News), TeamPCP's Trivy-to-LiteLLM-to-Telnyx chain (March 2026) (Akamai, LiteLLM), the Packagist compromises (April–May 2026) (Packagist Blog), Atomic Arch (June 2026) (TheNextWeb, BrainDetox), and the AUR adoption abuse (July 2026) (BleepingComputer) drew a convergent response: kill long-lived tokens, make versions immutable, add human

approval to automated publishing, add cooldowns, cut install-time code execution. npm's staged publishing (GitHub Changelog), Packagist's planned FIDO2 staged-release flow (Packagist Blog), and PyPI's 14-day file-rejection rule (PyPI Blog) are the same idea, reached on separate roadmaps.

Three capabilities remain almost untouched: reachability analysis (C5, where Go's govulncheck is the lone ✅ and the other fourteen have nothing (golang/vuln)), reproducible builds outside the distro world (X3), and registry-side cooldowns (X10). Composer's minimum-release-age policy, designed and gated on the version immutability that shipped in July 2026 (Packagist Blog, Packagist Blog), will be the first cooldown built as a registry-informed client policy rather than a Dependabot setting (BleepingComputer).

Funding attribution is legible and concentrated. Of the work catalogued here, Alpha-Omega paid for PyPI archival and the ZIP-confusion hardening (Trail of Bits, PyPI Blog), the RubyGems audit (RubyGems Blog), the Composer/Packagist audit (Packagist Blog), the crates.io threat model (crates.io threat model), the hex.pm audit (Hex Blog), and Homebrew's build provenance (Trail of Bits). The Sovereign Tech Agency/Fund paid for the Packagist transparency log and policy framework (Packagist Blog), PyPI's second audit (PyPI Blog), and part of PEP 740 (PyPI Blog). Open Technology Fund paid for the first PyPI audit (PyPI Blog) and the Homebrew audit (Homebrew). Google's OSS Security Team paid for PyPI trusted publishing and attestations (PyPI Blog, PyPI Blog); AWS funds PyPI's Safety & Security Engineer (PyPI Blog) and, through a Security Residency, funded RubyGems trusted publishing (RubyGems Blog); the OpenSSF funds the crates.io Security tab (Rust Blog) and PyPI's security-signal UI work (pypi/warehouse issue #19950). npm and NuGet stand apart, corporate-funded throughout. The pattern: grants paid for the L3 capabilities at each foundation-operated registry in this set, and the two registries that reached L3 on internal budgets are the two that hyperscalers own.

Sustainability

Security capability tracks money. The registries with the strongest L3 postures either have a corporate owner or hold recurring grants that pay for dedicated staff, and the registries with the thinnest postures run on volunteers.

The registry stewards said this themselves in September 2025. *Open Infrastructure is Not Free*, a joint statement published through the OpenSSF and signed by Alpha-Omega, the PSF, Ruby Central, the Rust Foundation, Sonatype, Packagist, the Perl and Raku Foundation, the OpenJS and Eclipse Foundations, and the CDF, describes registries serving billions of downloads a month on funding models built from goodwill: a small group of benefactors absorbing costs while commercial-scale consumers, CI systems, dependency scanners, and now AI-driven automation, extract value without contributing (OpenSSF et al). The statement names three directions for aligning usage with responsibility: commercial and institutional partnerships

proportional to usage, tiered access models that keep general use open while scaling performance for high-volume consumers, and value-added capabilities such as usage statistics that commercial users would pay for (OpenSSF et al). Six of the fifteen ecosystems in this document (PyPI, RubyGems, crates.io, Maven Central, Packagist, and CPAN) have a steward among the signatories, and the paid features catalogued below are the early, uneven implementations of exactly those three ideas.

This section covers how each registry funds its operation: who operates it, where the money comes from, whether the registry earns commercial revenue of its own, and whether a funder pays for dedicated security staff.

Registry	Operator	Primary funding	Own commercial revenue	Dedicated security staff
npm	GitHub (Microsoft)	Corporate budget	Paid private packages and Teams plans	GitHub security engineering (corporate)
PyPI	Python Software Foundation	Sponsorships, grants, donated CDN/infra	PyPI Organizations paid corporate tier, bringing in monthly income (PyPI Docs, PSF Blog)	Two Alpha-Omega-funded roles (PSF Blog)
NuGet	Microsoft	Corporate budget	Bundled into Microsoft's developer business	Microsoft security engineering (corporate)
RubyGems	Ruby Central	Sponsorships, grants, donated CDN/infra	None; sponsorship tiers up to $5,000/mo (The Register)	Alpha-Omega-funded engineer(s) in residence (Ruby Central, Alpha-Omega)
Packagist	Packagist Conductors (private company)	Private Packagist product revenue	Private Packagist funds the majority of the work; sponsorship tiers from €2,500/mo since Jul 2026 (Packagist Blog)	Company staff; Sovereign Tech Agency project funding (Packagist Blog)
Maven Central	Sonatype (private company)	Corporate budget, tied to Sonatype's repository product line	Indirect: Central anchors Sonatype's commercial Nexus business	Sonatype staff (corporate)

crates.io	Rust Foundation	Corporate memberships, grants, donated infra	None	Alpha-Omega-underwritten security team since 2022 (Rust Foundation, PR Newswire)
hex.pm	Hex core team (billing via Six Colors AB)	Private-organization subscriptions	$7/user/month for private packages ($70/yr annual) (Hex.pm, Hex.pm Blog)	None dedicated; Alpha-Omega funded the audit (Hex Blog)
Go modules	Google	Corporate budget	None; free services with a published privacy policy	Go security team (corporate) (golang/vuln)
Debian	Debian Project / SPI	Donations, donated mirrors and infrastructure	None direct; companies fund contributors through Freexian's commercial LTS offering (Freexian)	None dedicated; volunteer security team
Arch	Arch Linux project	Donations, donated infrastructure	None	None; volunteer
vcpkg	Microsoft	Corporate budget	Bundled into Microsoft's C++ tooling business	Microsoft staff (corporate)
Homebrew	Homebrew project	Donations (GitHub Sponsors et al.), donated CI/hosting from GitHub	None direct; pays maintainers a $300/mo stipend from donations (McQuaid); commercial adjacency via Workbrew, founded by the project leader (McQuaid)	None dedicated; Alpha-Omega funded provenance work (Trail of Bits)
CRAN	R Foundation / WU Wien and volunteer team	Academic institutional hosting, donated mirrors	None (Posit sells CRAN-adjacent products, none of which fund CRAN)	None; small volunteer team

CPAN	Volunteer (PAUSE admins) / Perl foundations	Donated hosting and mirrors	None	None; CPANSec is a volunteer group (CPAN Security Group)

A few of these deserve more than a row.

PyPI is the test case for registry-earned revenue. The PSF launched paid Organization accounts in 2023, free for community projects and subscription-priced for companies, with the stated goal of turning PyPI into a staffed platform (PyPI Blog, PyPI Docs). Uptake stalled for two years on staffing, and the PSF's October 2025 financial-outlook post is blunt about the wider picture: assets and revenue declined while costs and demand grew, corporate sponsorship slowed, and PyPI Organizations "has begun bringing in monthly income" as one of the few working diversification channels (PSF Blog). The registry that leads this document on Principles compliance runs on a nonprofit that describes its own finances as strained.

RubyGems shows what sponsor concentration costs. Ruby Central lost a quarter-million-dollar annual sponsorship in 2025, leaving it reliant on a small number of remaining corporate funders, and later that year a contested change in control of the RubyGems and Bundler repositories played out against that backdrop (The Register). This document takes no position on the governance dispute; the sustainability lesson stands on its own. A registry serving an entire language ecosystem operated with so few marginal funders that the departure of one reshaped its options, which is the funding-concentration risk the joint statement describes in general terms (OpenSSF et al). Ruby Central signed that statement the same month (OpenSSF et al).

Packagist and hex.pm run the commercial-subsidy model. Packagist Conductors funds packagist.org from Private Packagist product revenue and says so (Packagist Blog); hex.pm charges for private organizations and keeps the public registry free (Hex.pm, Hex.pm Blog). Both registries fund their public service from a paid private tier of the same product, the same shape as PyPI Organizations but a decade more established in hex.pm's case.

Paid features map onto the joint statement's three models. Of the statement's proposed directions (OpenSSF et al), the value-added and tiered ideas already exist in scattered form. Paid private hosting on shared registry infrastructure: npm private packages and Teams, hex.pm organizations (Hex.pm), Private Packagist (Packagist Blog), PyPI Organizations (PyPI Docs). Paid support and priority handling: PyPI offers priority support agreements to paid subscribers (PyPI Blog). Usage-proportional sponsorship with named benefits: the Composer/Packagist program from €2,500/month (Packagist Blog) and Ruby Central's tiers to $5,000/month (The Register). Commercial services adjacent to the registry that pay its people rather than its bills: Freexian's LTS offering, through which companies pay for extended Debian support and Freexian pays Debian contributors (Freexian), and the commercial Homebrew-management company founded by the project's leadership (McQuaid). What no registry in this set has shipped is the statement's second model applied to consumption: tiered or metered access for high-volume

downloaders. Every registry here still serves unlimited anonymous downloads, the CI and AI traffic the statement identifies as the cost driver rides on that, and the first registry to meter it will be the test case for the whole set.

Five registries are corporate line items. npm, NuGet, vcpkg, and the Go module infrastructure answer to Microsoft (the first three) and Google; Maven Central answers to Sonatype. Their security roadmaps need no grants, and it shows in the funding column of this document. The trade is dependency: their continued operation is a business decision by their owners, not a community guarantee. Maven Central functions as the free anchor for Sonatype's commercial repository business, and the Go ecosystem's proxy, checksum log, and vulnerability database all run on one company's willingness to keep paying for them.

The volunteer tier carries the most risk. Debian, Arch, CRAN, and CPAN run on donations, donated infrastructure, and volunteer time, with no revenue and no dedicated paid security staff. Debian compensates with the deepest structural controls in this document; CRAN and CPAN do not, and their positions at the bottom of Tables 1 and 2 line up with their positions at the bottom of this one. The correlation runs through the whole set: no registry here reached Authorization L3, or Go's equivalent architectural position, without either a corporate owner or a recurring external grant paying for the people who built it.

Closing: trends, learnings, directions

The self-service registries are converging on the curated model, minus the humans. Read the 2025–26 shipping lists side by side and the same shape appears everywhere: kill long-lived credentials, make published versions immutable, put a human approval step in front of automated publishing, delay adoption of brand-new versions, and stop running arbitrary code at install time. Debian has had each of those properties for decades, through keyrings, an immutable archive, the NEW queue, the testing migration delay, and package review. npm, PyPI, and Packagist are now rebuilding them one incident at a time, as staged publishing (GitHub Changelog), 14-day upload windows (PyPI Blog), version immutability (Packagist Blog), and script-off defaults (GitHub Changelog). The destination is a registry with curated-grade structural controls at self-service scale, and the registries that started with those controls by design, Go and NuGet above all, spent the last two years watching the rest of the field converge on their architecture.

Trust did not disappear; it moved, and the attackers moved with it. Trusted publishing shifts trust from maintainer credentials to CI identity, and the 2026 campaigns answered by attacking CI: the TeamPCP chain started in a GitHub Actions workflow (Akamai), and a captured OIDC token inherits the full trust of the workflow it came from. The registries saw this in their own designs, which is why crates.io removed the two most-abused workflow triggers from trusted publishing (Rust Blog) and why the Packagist team hardened its own pipelines with static

workflow analysis (Packagist Blog). The next control plane is the workflow itself: which triggers can mint identity, what the runner can reach, and whether a publish requires a human factor no runner holds. Registries that treat trusted publishing as the finish line will relearn this the hard way.

AI inverted the economics of discovery, and the response so far is people. Finding vulnerabilities and generating plausible malicious packages both got cheap in the same two years, so the scarce resource shifted from detection to verified judgment: someone has to separate the exploitable from the noise before a maintainer burns an evening on it. The staffing pattern that emerged is the clearest funding trend in this document. Alpha-Omega's residencies at the PSF, Ruby Central, the Rust Foundation, and the PHP Foundation now converge on that triage role (Rust Foundation, Alpha-Omega). The technical counterpart is reachability analysis, which cuts the noise before a human sees it, and the fact that only Go ships it (golang/vuln) while fourteen ecosystems field AI-assisted reports without it makes C5 the most obviously fundable gap in the capability tables.

The grant record supports a specific thesis: fund a capability once, in the right place, and it propagates. Trusted publishing was a funded PyPI project in 2023 (PyPI Blog); within three years RubyGems, crates.io, NuGet, and npm had adopted the design, and the OpenSSF WG turned it into cross-ecosystem guidance (OpenSSF Trusted Publishers). The audit-then-staff progression shows the same compounding: one-off audits surfaced findings, and the residencies that followed turned findings into shipped controls year over year. What grants have not fixed is anything structural about money. The funded registries still run on strained nonprofits, which is the sustainability section's whole argument, and a capability grant cannot buy a funding model.

Sustainability is becoming a security property, and consumption is the unpriced input. The joint statement's signatories named the problem (OpenSSF et al.), and the paid features catalogued above implement two of its three models. The third, tiered or metered access for high-volume consumers, remains unshipped anywhere in this set, while unlimited anonymous download traffic from CI and AI agents keeps growing as the cost driver. Some registry will meter first, the reaction will be loud, and every other steward will study the result. The adjacent risk is concentration: a registry with one marginal funder has a governance single point of failure, as 2025 demonstrated, and a registry with one corporate owner has an existence that rests on a business decision. Neither shape is resilient; the diversified-revenue shape that PyPI, Packagist, and hex.pm are building is the hedge.

Abandonment is the attack surface no framework covers. The AUR campaign worked by adopting orphaned packages and inheriting their accumulated trust, and the emergency response was to disable adoption outright, leaving roughly 13,000 orphans frozen in place (BleepingComputer). Each ecosystem in this document carries the same latent surface: packages with users and no maintainer, where whoever claims the name inherits its install base. The Principles have no capability for succession, and no registry has shipped one. A workable

answer needs the pieces this document keeps meeting separately: verified identity for the claimant, staged and reviewable takeover, cooldowns on post-adoption releases, and a transparency log that makes the change visible to consumers. Delegated or automated maintenance of abandoned packages is coming to these ecosystems one way or another; whether it arrives as an attack or as infrastructure depends on whether registries build the trust framework first.

Near-term directions worth watching. Mandatory MFA closes out across the holdouts: npm's last bypass path ends around January 2027 (GitHub Changelog), RubyGems committed to platform-wide expansion in 2026 (RubyGems Blog), and Packagist has set the direction with FIDO2 staged releases for large packages (Packagist Blog). Composer's minimum-release-age policy ships the first registry-informed cooldown now that immutability landed. Packagist's move to hosted artifacts with provenance will test whether a metadata registry can cross into Authorization L3. And the Principles need a v0.2: Table 5 is the candidate list, staged publishing and cooldowns are proven in production, maintainer-change visibility and post-removal downstream notification (X12 and X13, drawn from Nesbitt's questionnaire) are the two capabilities almost no registry has built and none is scored on, and the framework should also learn to score the account-less architecture Go demonstrates, which today earns N/A for properties that are among its strongest security decisions.

Caveats

- **Ratings are judgment calls.** ◐ covers a range: "exists but opt-in," "exists via third-party tooling," "equivalent by other means." Where a rating turns on interpretation, the per-registry text says so.
- **The tables score existence, not adoption.** A ✅ means a capability is shipped and available, not that most packages use it. An opt-in attestation on three percent of a registry's packages and a mandatory one on all of them both read as present here, and the security difference between them is large. Where coverage is known it is stated in prose (npm's default-on lifecycle-script change, RubyGems' MFA threshold reaching about 370 of 190,000+ gems, Homebrew's attestation verification being off by default), but the symbols themselves do not carry it. Read a ✅ as "the operator built this," and check the per-registry text for how widely it applies.
- **The Principles are v0.1 (Feb 2024)** (OpenSSF Principles) and predate trusted publishing's spread, staged publishing, install-time script controls, and cooldown policies. Table 5 exists because the framework has not caught up; several of its columns are candidates for a v0.2.
- **The picture moves fast.** GitHub shipped npm changes on 28 and 31 July 2026 (GitHub Changelog, GitHub Changelog); Arch disabled AUR adoption on 30 July 2026 (BleepingComputer). Re-verify anything dated within the last quarter before publication.

The *Open Infrastructure is Not Free* statement (OpenSSF et al) promises follow-up work and additional signatories; this document reflects the statement as published and did not locate a subsequent letter, so check for one before publication.

- **Cells not confirmed by a primary source.** Most registries document A6 (abandoned-domain detection) and A9 (leaked-credential checks) thinly; the ratings there for npm, NuGet, RubyGems, Maven Central and hex.pm rest on inference from platform behavior (GitHub- or Microsoft-account inheritance, for example) rather than on an announcement. NuGet's Z6 ◐ and Maven Central's G5 ◐ also rest on partial public information. The Debian and Arch A-track ratings reflect distribution account practices (Salsa/SSO, keyring policies) that no single document describes as registry features. In the sustainability table, the "donated CDN/infra" entries for PyPI and RubyGems reflect the in-kind sponsorships (Fastly, AWS, and others) that both registries acknowledge on their sponsor pages, which this document does not cite as individual sources. Treat these cells as lower-confidence.

Related work

This document scores the registry: what an operator has built. The natural complement is Andrew Nesbitt's package-manager threat-model questionnaire (Nesbitt), which frames the same territory as open questions to answer per tool, and whose unit is the package *manager* (client plus registry plus the tool's own supply chain) rather than the registry alone. Several of its questions (install-time and pre-install code execution, lockfile hash-verification ordering, maintainer-change visibility, downstream notification of pulled packages) sit below or beside the granularity of a capability matrix; two of them are folded into Table 5 as X12 and X13, and the design-level ones are noted where they arise. Read his questionnaire for the client-side and design questions, and this matrix for who has shipped which answers.

Change Log

- Original assessment date: 3 August 2026.
- Post-assessment validation: 30 September 2026. The OpenSSF Principles remain at v0.1, and no post-August development identified in this review requires a change to the matrix symbols.
- A September 2026 validation found that the comparative conclusions remain broadly sound. npm strengthened trusted and staged publishing; crates.io experienced a real maintainer-compromise incident that validated several gaps already identified in this report; and Maven Central introduced a paid Publisher Pro model for some commercial and higher-volume publishers. These developments update the evidence and sustainability narrative but do not, on the available public evidence, require changes to the capability ratings.